

```

1: function CS_TSSOS_FIRST(pop, x, d, keyword arguments)
2:   ▷ obtain more efficient way to represent monomials
3:   n, supp, coe ← polys_info (pop, x)
4:
5:   ▷ Merge monomials with same support in objective
6:   supp[1], coe[1] ← resort (supp[1], coe[1])
7:
8:   if clique is not specified then
9:     cliques ← clique_decomp (n, m, numeq, dc, supp)
10:
11:   ▷ Group Constraints according to variables and cliques
12:   I, J, ncc ← assign_constraint (m, numeq, supp, cliques, cql)
13:
14:   ▷ Determine max degree of monomials in subproblem for each clique
15:   if d is “min” then
16:     rlorder ← max degree in I, J for each clique
17:   else
18:     rlorder ← d for all cliques
19:
20:   if custom basis not specified then
21:     ▷ If Correlative Sparse, need to loop over all cliques
22:     basis ← get_sbasis (cliques, rlorder)
23:     ▷ Why is this sparse?
24:     hbasis ← get_sbasis ()
25:   if Term Sparsity is required then
26:     ksupp ← supp + square of all terms in supp
27:     ▷ Apply Term Sparsity by chordal extension, We can improve this
28:     blocks, eblocks, cl, blocksize ← get_blocks (I, J, supp, cliques, cql, ...)
29:     opt, ksupp, moment, GramMat ... ← solvesdp (n,m,...)
30:     ▷ Construct the data structure for higher order improvement
31:     data ← mcpop_data (n, nb, m, numeq, ...)
32:     ▷ Why is this step necessary?
33:     if solution is true then
34:       sol, gap, data.flag ← approx_sol (n,m,...)
35:     return opt, sol, data

```

```

1: function CS_TSSOS_FIRST(A, n, v)
2:   ▷ Initialize the search range
3:   l ← 1
4:   r ← n
5:
6:   while  $l \leq r$  do
7:     mid ← floor ( $\frac{l+r}{2}$ )
8:     if  $A[\text{mid}] < v$  then
9:       l ← m + 1
10:    else if  $A[\text{mid}] > v$  then
11:      r ← m - 1
12:    else
13:      return m
14:   return null

```

```

1: function TSSOS_FIRST( $A, n, v$ )
2:   ▷ Initialize the search range
3:    $l \leftarrow 1$ 
4:    $r \leftarrow n$ 
5:
6:   while  $l \leq r$  do
7:      $\text{mid} \leftarrow \text{floor}(\frac{l+r}{2})$ 
8:     if  $A[\text{mid}] < v$  then
9:        $l \leftarrow m + 1$ 
10:    else if  $A[\text{mid}] > v$  then
11:       $r \leftarrow m - 1$ 
12:    else
13:      return  $m$ 
14:  return null

```

matrixsos.jl LinearPMI_first